Q10. Intelligent vs Creative Agent and Search, and Expanding the Action Space with RL

> **Question.** Explain intelligent agent and creative agent, intelligent search and creative search. If a creative agent must expand its search space and action space, how can we write a function for it? Take product design with a utility-based agent and reinforcement learning to expand the action space. Explain different options.

1. The idea in plain words

An **intelligent** agent is very good at reaching a goal with the moves it already has. A **creative** agent also asks: "*Do I have the right moves at all?*" If the answer is no, it invents new moves. In product design this means the agent does not only pick features one by one. It notices which feature *pairs* work well together and turns each pair into a single new action. The agent's action space grows, so the space of designs it can reach grows too.

2. Intelligent agent vs creative agent

	Intelligent agent	Creative agent
Aim	Rational: maximise the performance measure	Rational **but non-obvious**: also novel and useful
Structure	Agent = architecture + program	+ **creativity calibrator** and **conceptual space expander**
PEAS	Fixed P, E, A, S	Asks "what could be different?" for P, E, A and S
Actions	Fixed action set	Creates new actions (exploration, analogy, combination)
Learning	Improves its policy	The learn set evolves, so the algorithm evolves, so the outcome evolves

3. Intelligent search vs creative search

	Intelligent search	Creative search
Space	Search the given space efficiently	Create, discover or invent new options
Neighbours	Fixed neighbour definition	Redefines the neighbour
Nodes	Expands existing nodes	Combines two or more nodes (merging states)
Actions	Uses the action set as given	Creates new sets of actions
Goal	Fixed goal test	Goals may change (dynamic goals)
Result	Obvious best answer	Useful, surprising choices

4. Assumptions

- A phone has 6 slots (screen, battery, camera, body, sensors, AI) with 4 options each.
- **Basic action** = fix one slot to one option, giving 6 × 4 = **24 actions**. The agent has only **3 steps** per design.
- **Utility** = sum of feature quality + hidden **synergy bonuses** for 3 feature pairs (for example the right body with the right screen). The agent does not know the pairs. It must discover them.

- The agent is a utility-based learner using epsilon-greedy Q-learning. It runs 1,500 design episodes and expands its actions at episode 500.

5. A function that expands the search and action space

The idea: look at the best designs built so far and find **action pairs that keep appearing**. Each pair becomes a new macro-action that sets two slots in one step. With 3 steps, the agent can now fill up to 6 slots, so designs that were unreachable become reachable.

```
function expand(actions A, best_runs R):
    pairs ← count every consecutive pair (a_i, a_{i+1}) in the top runs R
    for each pair (a, b) among the top-k most frequent:
        if (a, b) not in A:  A.add( macro(a, b) )          # new action = a then b in one step
    return A                                                  # bigger action space, bigger
search space

function CreativeRL(A, episodes, warmup):
    for ep = 1 to episodes:
        if ep = warmup + 1: A ← expand(A, top 60 runs);  reset Q
        design ← run epsilon-greedy episode using actions in A
        reward ← utility(design)
        update Q(state, action) toward reward
    return Q, A
```

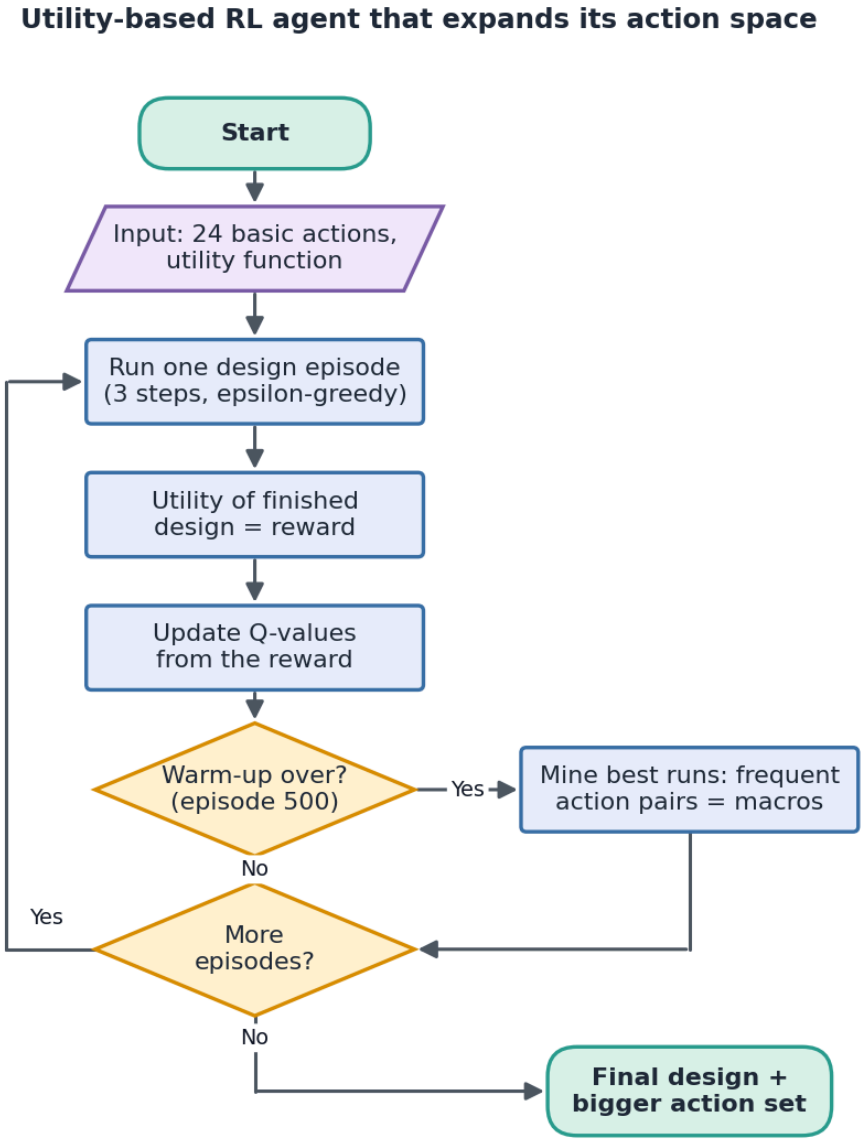


Figure 1. Flowchart of the utility-based RL agent that expands its action space.

6. Step-by-step approach

1. Start with 24 basic actions and the utility function (the agent's "happiness" measure).
2. Run design episodes. Each has 3 steps. The agent picks mostly the best-known action and sometimes a random one.
3. Score the finished phone with the utility function and update the Q-values with that reward.
4. After the warm-up (episode 500), look at the 60 best runs and mine the most frequent action pairs.
5. Add them as macro-actions. Clear the old Q-values, because the action list has changed.
6. Keep learning with the bigger action set. Compare with an agent whose actions never grow.

7. Worked example (real run)

The agent found these four macro-actions, which raised the action space from **24 to 28**:

New macro-action	Is it a hidden synergy?
body = 2 + screen = 1	Yes (bonus +2.0)
battery = 2 + sensors = 1	Yes (bonus +2.5)
screen = 1 + ai = 2	No, a lucky pair
body = 2 + battery = 2	No, a lucky pair

Two of the four were real synergies the agent had never been told about. Utility before and after the expansion:

Episodes	Fixed 24 actions	With expansion
1–500 (same for both)	1.92	1.92
501–1000	2.92	3.13
1001–1500	2.98	4.68
Final greedy design	**3.11**	**6.46**

The fixed agent stalls near 3 because three single-feature steps cannot reach the synergies. The creative agent ends **about 2 times higher** (6.46 vs 3.11) with the same three steps. Note the short dip just after episode 500 in Figure 2: we clear the Q-table when the actions change, so the agent has to relearn. The extra learning cost pays off later.

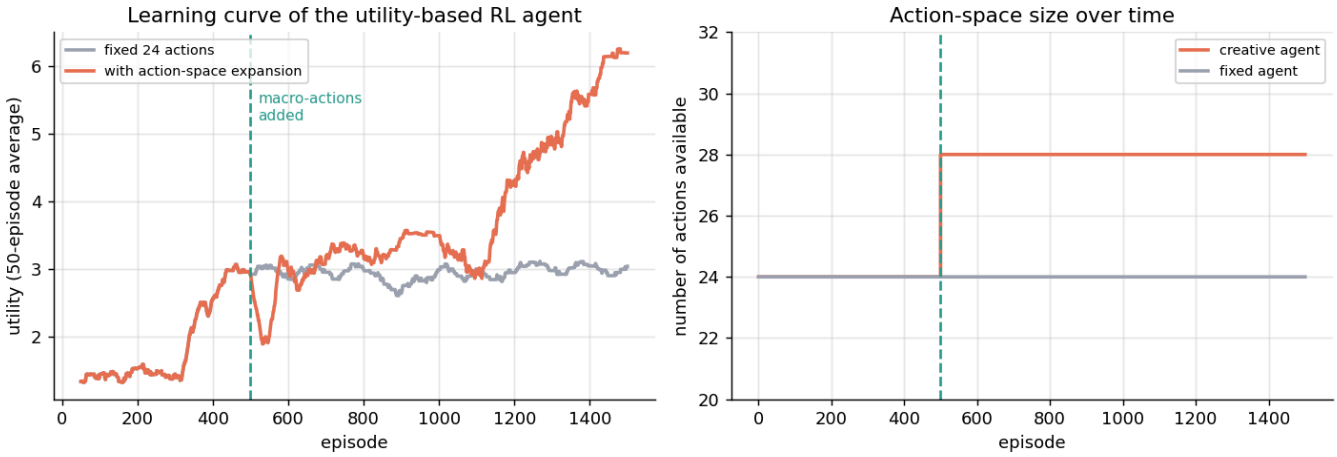


Figure 2. Left: utility per episode (50-episode average). Right: number of actions over time.

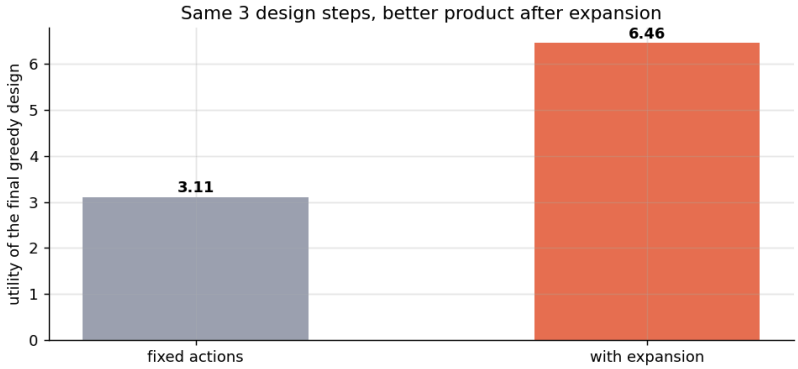


Figure 3. Utility of the final greedy design.

8. Different options to expand the action space

Option	What it does	Product design example
Macro-actions (options)	Turn frequent successful sequences into one action. *Used in our run.*	"Fold-able body + flexible screen" as one step
Parameterised actions	Make an action continuous instead of a fixed choice	Battery size from 3,000 to 6,000 mAh
Analogy / transfer	Copy actions that worked in another domain	Camera stabiliser ideas from drones
Human co-creator	A designer suggests a new action; the agent tests it	"Add a solar back panel"
Curiosity reward	Give extra reward for reaching new, unseen states	Tries rarely used feature mixes
GA over action sequences	Evolve sequences with crossover and mutation, keep the fittest as new actions	Mix two good build orders
New sensor / modality (IoT)	A new sensor adds a dimension to the space	Add a gas or health sensor slot

9. Code and real run

`q10_code.py` trains both agents, mines the macro-actions and saves the charts.

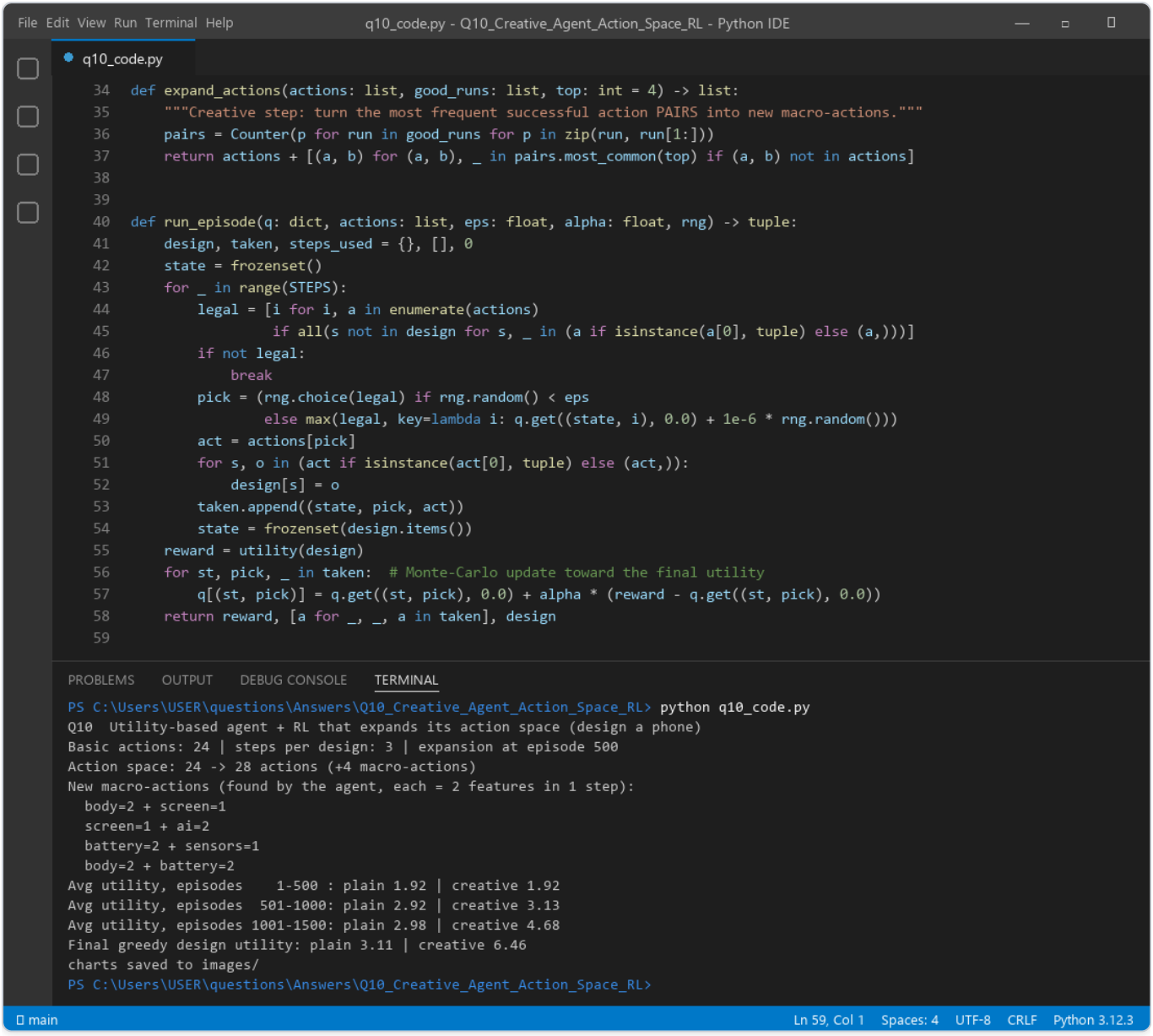


Figure 4. The `expand_actions()` and `run_episode()` functions with the real terminal output.

10. Summary

> An intelligent agent searches well inside the space it is given. A creative agent also changes the space: it creates new actions and redefines neighbours. By mining its best action pairs into macro-actions, the RL agent grew its action set from 24 to 28 and nearly doubled its final utility (3.11 to 6.46) in the same three steps. Other options are parameterised actions, analogy, human co-creators, curiosity and genetic search over action sequences.

Files in this folder: `q10_code.py` (source), `q10_code.ipynb` (Colab/Jupyter version, already run), `q10_output.txt` (console output), `images/` (all figures).

Q9. Psychological Creativity through Evolutionary Algorithms, and the Step to Historical Creativity

> **Question.** Discuss different possibilities where psychological (P) creativity is achieved through an evolutionary algorithm. Can it be extended to historical (H) creativity? What could the challenges be? Think of a few ideas to resolve them. How can we expand the conceptual space in this case?

1. The idea in plain words

An evolutionary algorithm (EA) keeps making new designs by mixing and mutating old ones. Almost everything it makes is **new to itself**. That is P-creativity (Boden): new to the individual or agent. H-creativity is much harder: the idea must be **new to all of history** and still useful. The EA cannot see history unless we show it. So the honest answer is: yes, it can be extended, but only if we (1) give the EA a memory of what the world has already made and (2) let it step out of the old conceptual space.

2. How we measure the two kinds of creativity

Kind	Test used in this answer
P-creative	Distance to the nearest design in the agent's **own archive** > 0.12, so the agent has never made anything like it
H-creative	Distance to the nearest of **300 historical designs** > 0.40 **and** usefulness > 0.70. It is new to the world *and* valuable

Every H-creative design is also P-creative, but not the other way round. In other words, P-creativity is H-creativity judged against a much smaller archive.

3. Possibilities where P-creativity comes from evolution

Area	What evolves	Why it is P-creative
Vehicle / product design	Body and wheel genes (the car GA from the lectures)	Crossover gives body/wheel mixes the agent never drew before
Poetry	Word sequences (POEVOLVE, McGonnagall)	New lines that fit rhyme and metre
Music	Jazz riffs (Al Biles' GenJam)	Each solo differs from the bot's earlier ones
Chatbots and rules	Decision trees (see Q1)	New dialogue strategies for the bot
Neural networks	Connection matrices, learning rules (Chalmers 1990)	New architectures for that system
Art and patterns	Drawing programs (see Q8)	New shapes from combined actions

4. Assumptions for the experiment

- A design is a small vehicle with 6 genes in [0, 1]: length, height, wheel size, wheel gap, roof slope and colour.
- **Usefulness** checks simple engineering rules: wheels near the ends, wheels sized to the body, roof not too steep.

- **History** = 300 known designs around 5 prototypes (sedan, SUV, truck, bus, sports car).
- EA: 40 designs per generation, 40 generations, tournament selection, blend crossover, Gaussian mutation, 2 elites.
- Three runs, all with the same seed:
  - **P-only**: fitness = usefulness + novelty against the agent's own archive.
  - **global**: from generation 21, fitness also rewards distance to history (a global archive).
  - **global+expand**: as "global", plus a 7th gene from generation 21, a "smart IoT skin" (sensor coverage) that no historical design has.

5. Step-by-step approach

1. Load the history archive H and start with an empty own archive P.
2. Create a random population and score each design: fitness = usefulness + novelty.
3. Label each design: P-creative if far from P (then store it in P), H-creative if also far from H and useful.
4. If the H-creative rate stays low, apply the fixes: reward distance to H, and expand the conceptual space with a new gene.
5. Select, cross over and mutate to make the next generation. Repeat until the last generation.

Evolutionary loop with P- and H-creativity checks

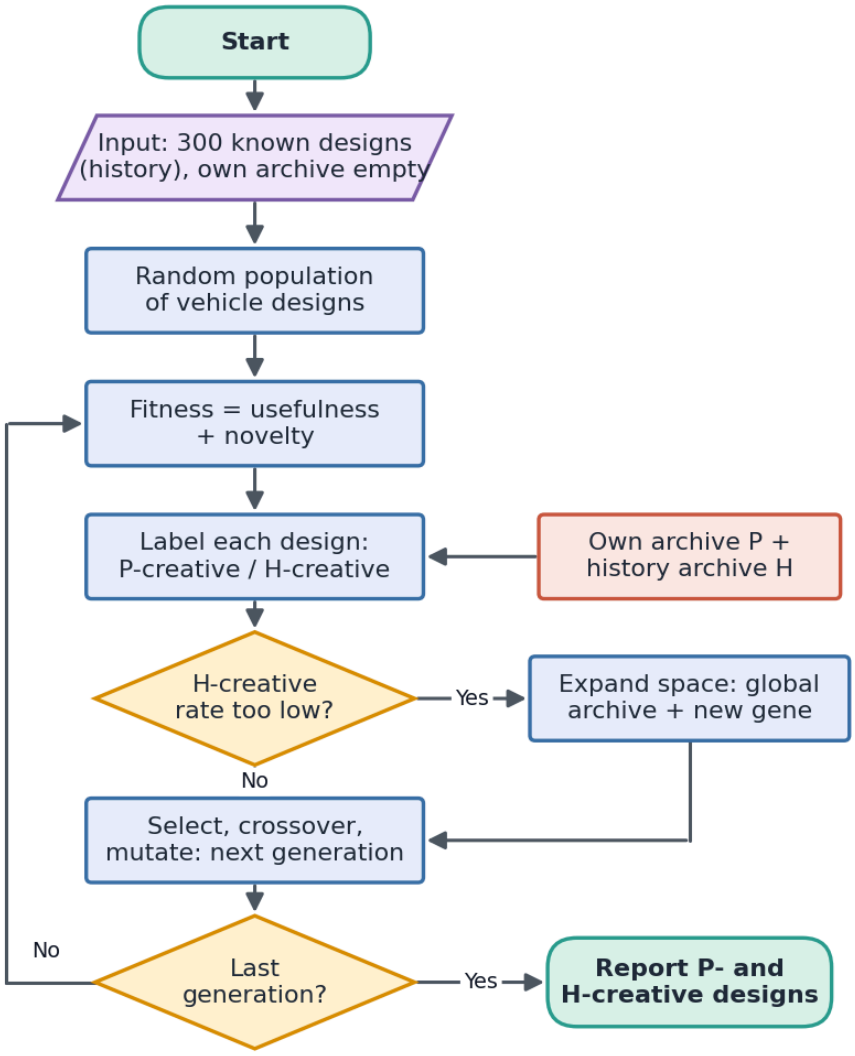


Figure 1. Flowchart of the EA with P- and H-creativity checks.

6. Pseudocode

```
function CreativeEA(history H, generations G, switch S):
    P ← { };  population ← random designs
    for g = 1 to G:
        if g = S + 1: add a new gene to every design            # expand the conceptual space
        for each design x in population:
            if dist(x, P) > τ_P:
                P.add(x)                                        # P-creative
                if dist(x, H) > τ_H and useful(x) > 0.7: mark x H-creative
        fitness ← useful(x) + 0.5·novelty(x, P)
        if g > S: fitness ← useful(x) + 0.3·novelty(x, P) + 0.6·novelty(x, H)   # global archive
        population ← elites + mutate(blend_crossover(tournament(population)))
    return P-creative and H-creative designs
```

7. Worked example (real run)

Results of the "global+expand" run, in blocks of 5 generations:

Generations	P-creative	H-creative	H share	What is happening
1–5	130	11	8.5%	Everything is new to a "newborn" agent
6–10	46	3	6.5%	Own archive fills up
11–15	21	0	0.0%	Useful designs drift back toward history
16–20	15	1	6.7%	Stuck: P-creative but not H-creative
21–25	60	12	20.0%	Fixes switched on
26–30	60	26	43.3%	The new gene is being explored
31–35	**47**	**37**	**78.7%**	**Most new designs are now new to history**
36–40	42	30	71.4%	Stable

Totals after 40 generations for the three runs:

Run	P-creative	H-creative	H share
P-only (plain EA)	276	18	6.5%
+ global archive	294	33	11.2%
+ global archive + new gene	**421**	**120**	**28.5%**

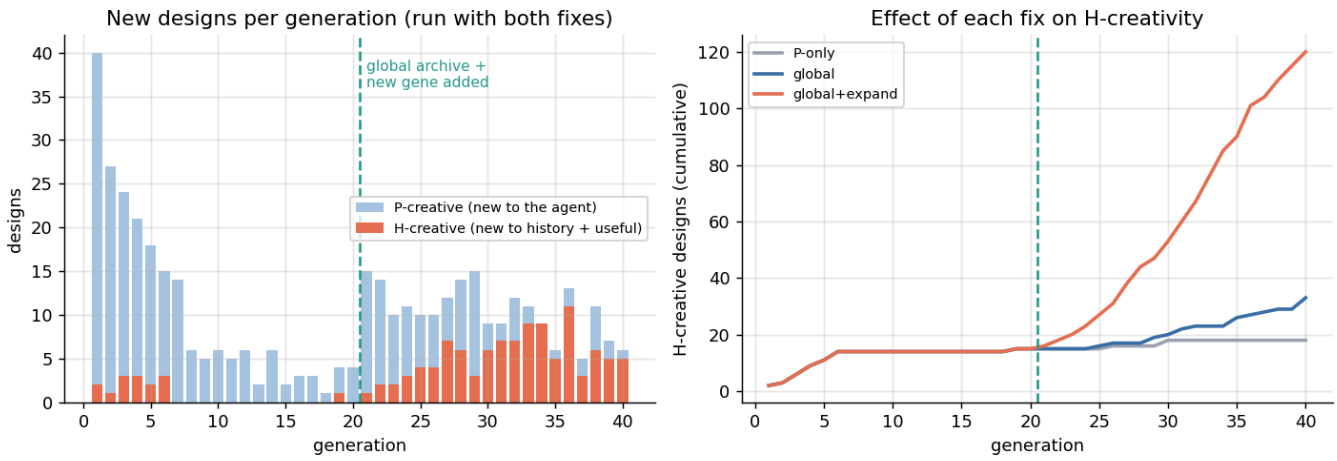


Figure 2. Left: P-creative (blue) vs H-creative (orange) designs per generation. Right: cumulative H-creative designs for the three runs.

8. Can it be extended to historical creativity?

Yes, but not for free. The plain EA made 276 P-creative designs, yet only 18 (6.5%) were new to history and useful. The reason is simple: the useful region of the old space is already crowded with sedans, SUVs and trucks, so evolution keeps "rediscovering" them. Once the EA could see history (global archive) and had a new dimension to explore, it produced **120 H-creative designs, 6.7 times more**. Most of them sit where no known vehicle sits: short but tall bodies (Figure 3, right).

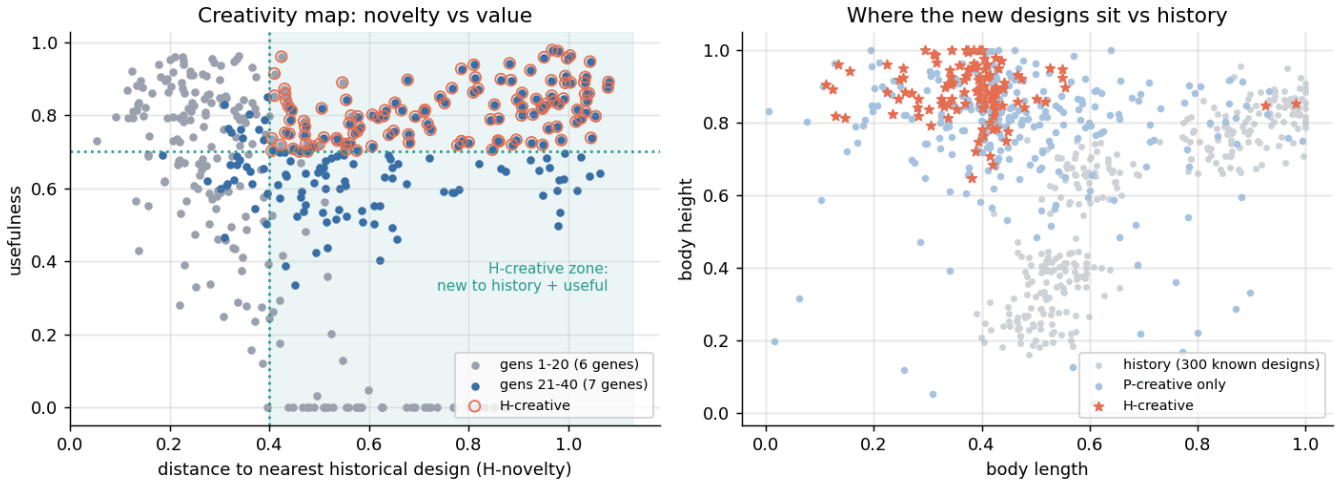


Figure 3. Left: novelty vs usefulness; the shaded corner is the H-creative zone. Right: new designs vs the 300 historical designs (grey).

9. Challenges

- **Knowing history.** No archive holds every design ever made, so an "H-creative" label is only as good as the archive.
- **Judging value.** Usefulness for brand-new ideas is uncertain. History often decides value later (a social judgement).
- **Novelty vs usefulness.** Random designs are novel but useless (the grey points at usefulness 0 in Figure 3).
- **Pull back to the known.** Fitness rules are written by people, so they favour what already exists (generations 11–20).
- **Premature convergence** and homogenisation. AI can raise individual creativity yet reduce collective diversity.

- **Cost.** Comparing every design with a large history archive and validating with experts is slow.

10. Ideas to resolve them

Idea	How it helps
Global archive + novelty search	Reward distance to *history*, not just to the agent's memory (+15 H-creative in our run)
Multi-objective fitness	Keep novelty, usefulness and surprise as separate goals (a Pareto front) so neither hides the other
Island models	Several sub-populations evolve apart and swap genes now and then, which keeps diversity
Human-in-the-loop (creativity test)	Experts rate candidates in the H-creative zone. The machine proposes; people judge value
Embedding search over patents/papers	A practical way to build a big history archive
Transformational operators	Change the representation itself, not only gene values (next section)

11. How can we expand the conceptual space?

- **Add a new dimension (gene).** In our run, a "smart IoT skin" gene appeared at generation 21. No historical vehicle has it, so useful designs that use it are new to history. The H share jumped from about 7% to **79%**. This is the course idea: *adding a sensor modality changes the space, not merely the data*.
- **Release boundaries of compatibility.** Relax rules such as "exactly two axles", like the pink-elephant overrule.
- **Change the rules R and traversal T** (Wiggins). Allow new operators such as nesting or analogy from other domains.
- **Combine spaces from other domains** (systemic view): vehicle + drone, or vehicle + furniture.
- **Generalise properties** as in the "GENERATE" slide. "Some shape on two circles" invites a pentagon on wheels.

12. Code and real run

`q09_code.py` runs all three EAs, counts P- and H-creative designs and draws the figures.

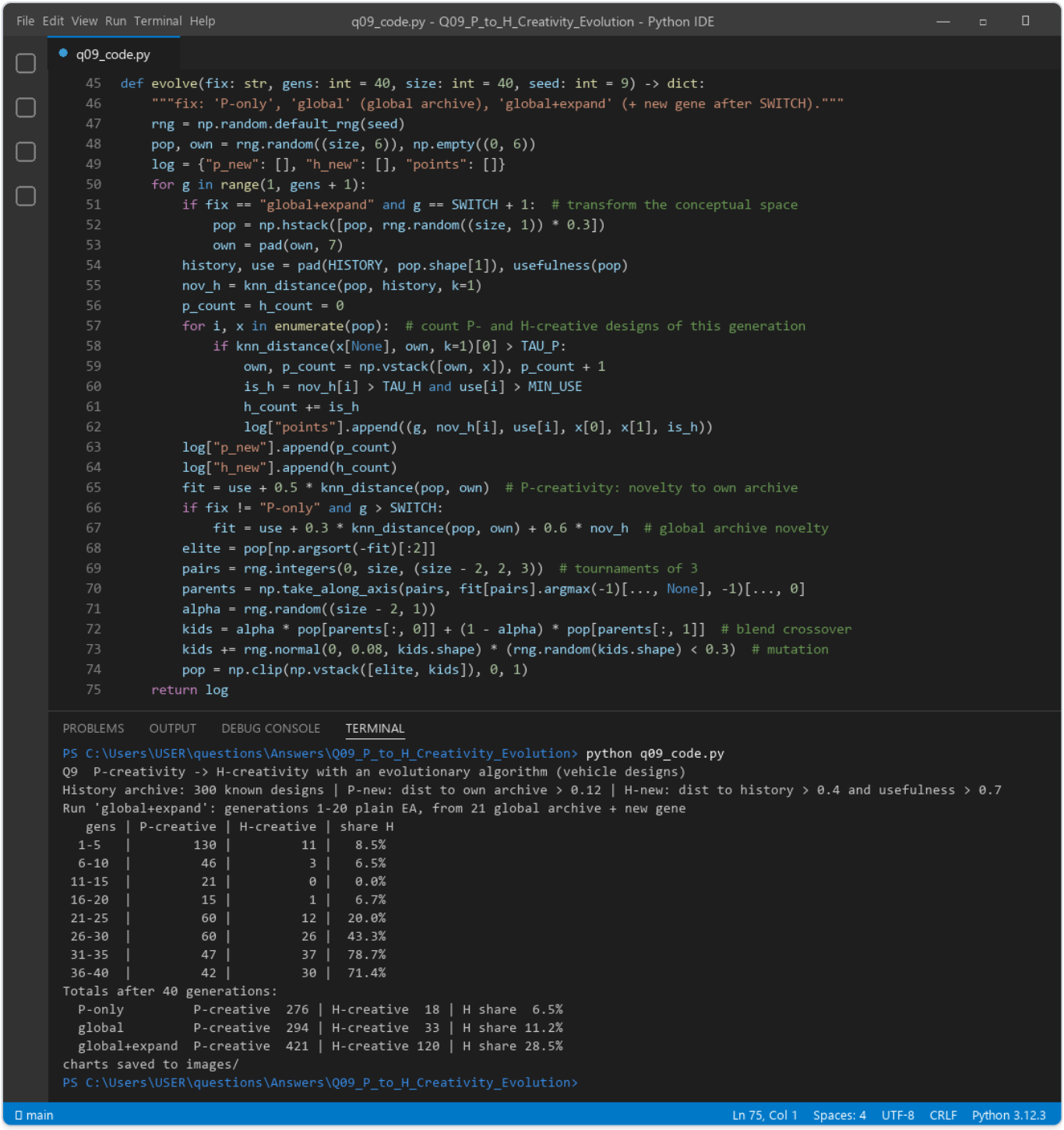


Figure 4. The `evolve()` function and the real terminal output.

13. Summary

> Evolution gives P-creativity easily: 276 designs new to the agent. H-creativity needs more, because only 6.5% of those were new to history and useful. Giving the EA a global archive and expanding its conceptual space with a new dimension raised that to 120 H-creative designs. Final value judgement should still involve people.

Files in this folder: `q09_code.py` (source), `q09_code.ipynb` (Colab/Jupyter version, already run), `q09_output.txt` (console output), `images/` (all figures).

Q8. Search Space and a Multi-Agent System that Combines Two Drawing Patterns

> **Question.** What is a search space? Design a multi-agent system that combines two drawing patterns into a new creative drawing, by expanding the search space or adding a new action. How can we combine two actions? Both agents introduce new combinations to create new states. What could the fitness function be?

1. The idea in plain words

Think of two artists. One only draws square spirals; the other only draws stars. Each artist knows a few moves: go forward, turn, make the next step longer. Neither can draw anything new with their own moves alone. The trick is to **glue one move from each artist into a brand-new move**, for example "turn like the star *and* grow like the spiral" in a single step. That new move opens up drawings neither artist could reach before. Two agents keep proposing such combinations, a fitness function scores them, and a shared archive remembers what is already done.

2. What is a search space?

The **search space** is the set of all states an agent can reach from the start state by applying its actions (operators), again and again. It is an implicit graph: states are nodes and actions are edges. We never build it fully; the search algorithm explores a small part of it. Two things decide its size:

- **The action set.** More actions means more edges out of every state.
- **The depth.** With a actions and sequences of length L, there are a^L possible sequences.

Here a state is a **drawing program** (a motif of actions repeated n times). Adding new actions therefore expands the search space directly. That is the creative lever in this question.

3. Assumptions

- Drawings are made by a "turtle" pen. The basic actions are `F` (forward), `L(θ)` (turn left), `G(g)` (make the step longer) and `C` (change colour).
- **Agent A** owns the square spiral: repeat [F, L(90°), G(3)] 40 times.
- **Agent B** owns the five-point star: repeat [F, L(144°)] 5 times.
- The agents work in rounds (6 rounds, 5 proposals each per round) and share one archive of "already seen" drawings.

4. Design of the multi-agent system

Part	In this design
Agents	A-spiral (likes compose, blend, nest) and B-star (likes compose, interleave, nest)
State	A drawing program: one motif of actions repeated n times (n = 20–70)
Basic actions	F, L, G, C (4 actions)
New actions	`LG` = turn + grow in one move, `STAR` = whole star as one move, `BLEND` = mixed turn angle (now 7 actions)
Agent behaviour	60% of the time an agent tweaks its own best idea (exploit); otherwise it tries a fresh combination (explore)
Shared archive	Stores every drawing that is clearly new (overlap with all archived drawings below 60%)
Goal	Highest fitness after 6 rounds, plus a set of diverse new drawings in the archive

5. How can we combine two actions?

Way	What it means	Example from our run
Compose	Run one action right after the other and treat the pair as one new action: $a_2 \circ a_1$	LG = L(144°) from the star + G(3) from the spiral. The result is a growing star-spiral.
Blend	Mix the parameters of two similar actions	BLEND turn = $\alpha \cdot 90° + (1-\alpha) \cdot 144°$. With $\alpha = 0.5$ that is 117°.
Nest	Pack one whole pattern into a single action and use it inside the other	STAR is drawn at every corner of the spiral (two levels of scale).
Interleave	Alternate the two parents' moves	One spiral move, then one star move, and so on.

Every way creates a **new action**. The action space grows from 4 to 7. For a motif of 6 actions the search space grows from 4^6 = 4,096 to 7^6 = 117,649 programs: **28.7 times larger** (Figure 3, right). This is exactly how adding an action expands the search space.

6. Step-by-step approach

1. Give each agent its parent pattern and the four basic actions.
2. In each round, every agent picks one action from each parent and combines them (compose, blend, nest or interleave).
3. Repeat the new action n times to get a drawing, which is a new state.
4. Score it with the fitness function.
5. If it is clearly different from everything in the shared archive, add it. Both agents now avoid it.
6. Each agent keeps its best drawing and, next round, either tweaks it or tries a fresh combination.
7. After the last round, show the best drawings as *choices* to the human designer.

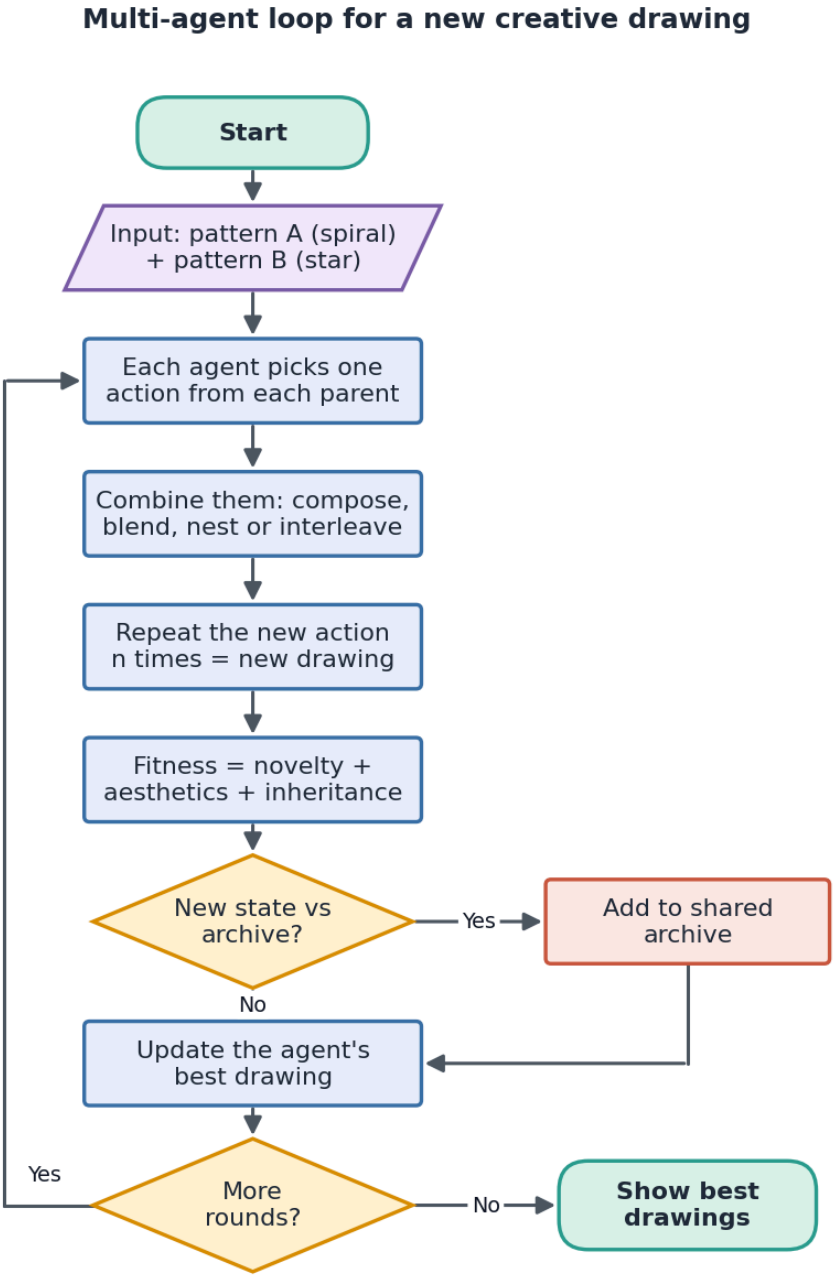


Figure 1. Flowchart of the multi-agent drawing system.

7. Pseudocode

```
function MultiAgentDrawing(parentA, parentB, rounds, k):
    archive ← { };  best[agent] ← none for both agents
    for r = 1 to rounds:
        for each agent in {A-spiral, B-star}:
            repeat k times:
                if best[agent] exists and random() < 0.6:
                    candidate ← combine(best[agent].operator, mutate(best[agent].params))
                else:
                    op ← random choice from agent's favourite operators
                    candidate ← combine(op, random params)          # new action
                drawing ← run_turtle(candidate)                         # new state
                score ← fitness(drawing)
                if overlap(drawing, every archived drawing) < 0.6: archive.add(drawing)
                if score > best[agent].score: best[agent] ← candidate
    return best drawings + archive
```

8. What could the fitness function be?

Fitness = 0.4 × Novelty + 0.4 × Aesthetics + 0.2 × Inheritance

- **Novelty** = 1 − the largest overlap (IoU of the pixel grids) with either parent. A drawing that looks like a plain spiral or a plain star scores low.
- **Aesthetics** borrows from Alexander's *living geometry*. It combines:
  - **local symmetry** (does the drawing match itself after a 3-, 4-, 5-, 6- or 8-fold rotation?);
  - **balanced coverage** (about 30% of the canvas inked);
  - **levels of scale** (a healthy mix of short and long strokes).
- **Inheritance** = $1 - |2 \cdot \text{share}_A - 1|$. It is 1.0 when both parents contribute equally, so the child is a real *combination* and not a copy of one parent.

9. Worked example (real run)

Both parents start with an aesthetics score of only **0.540** (spiral) and **0.547** (star). The table shows each agent's best drawing so far after every round:

Round	Agent	Best operator	Fitness	Novelty	Aesthetics	Archive size
1	A-spiral	compose	0.703	0.732	0.525	2
1	B-star	nest	0.616	0.499	0.641	3
3	A-spiral	compose	0.779	0.684	0.763	10
3	B-star	compose	0.735	0.677	0.661	13
6	**A-spiral**	**compose**	**0.793**	**0.672**	**0.810**	**23**
6	B-star	compose	0.749	0.669	0.703	26

Check of the winner: 0.4 × 0.672 + 0.4 × 0.810 + 0.2 × 1.0 = **0.793**. B-star started with "nest", then switched to "compose" after it saw that operator do better. By the end the archive held **26 different new drawings**.

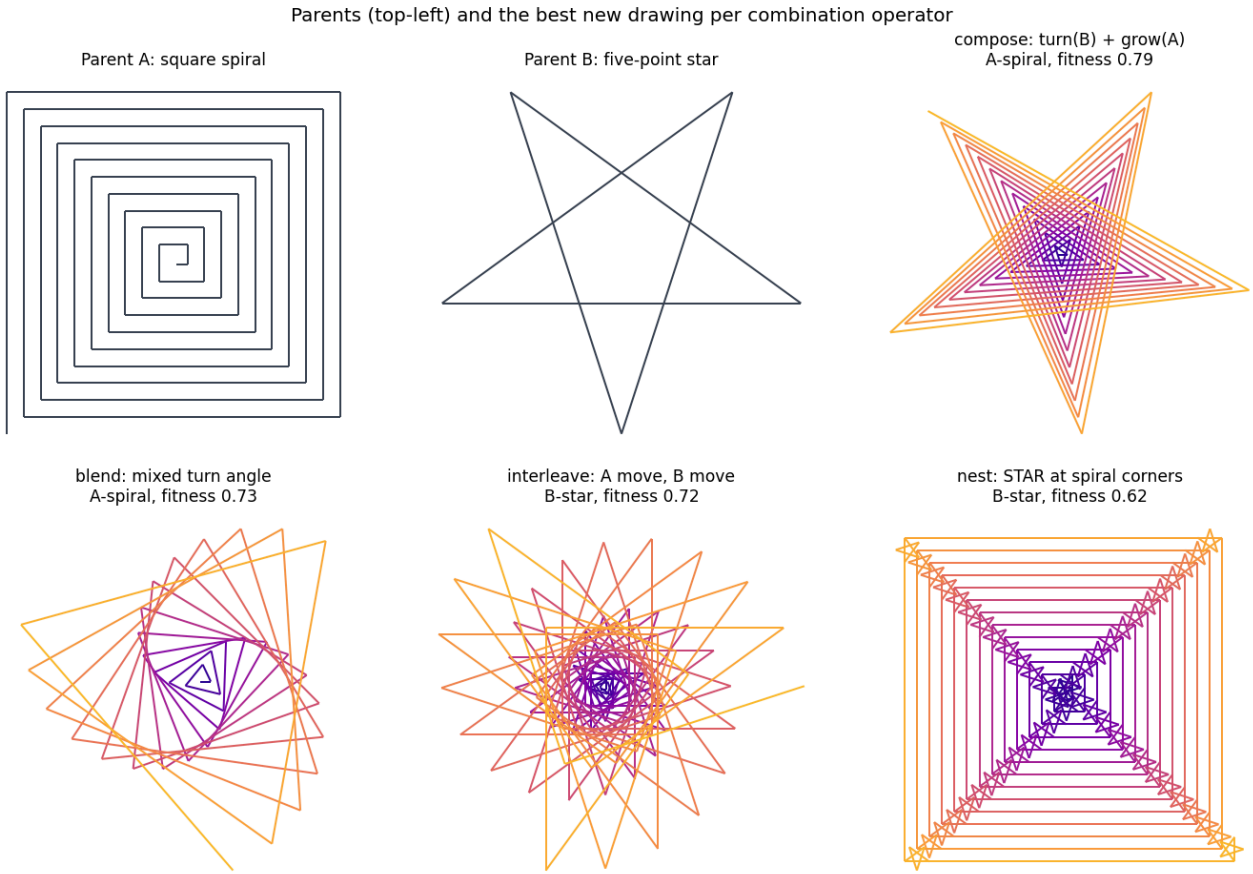


Figure 2. The real drawings. Top-left: the two parents. Others: the best new drawing for each way of combining actions.

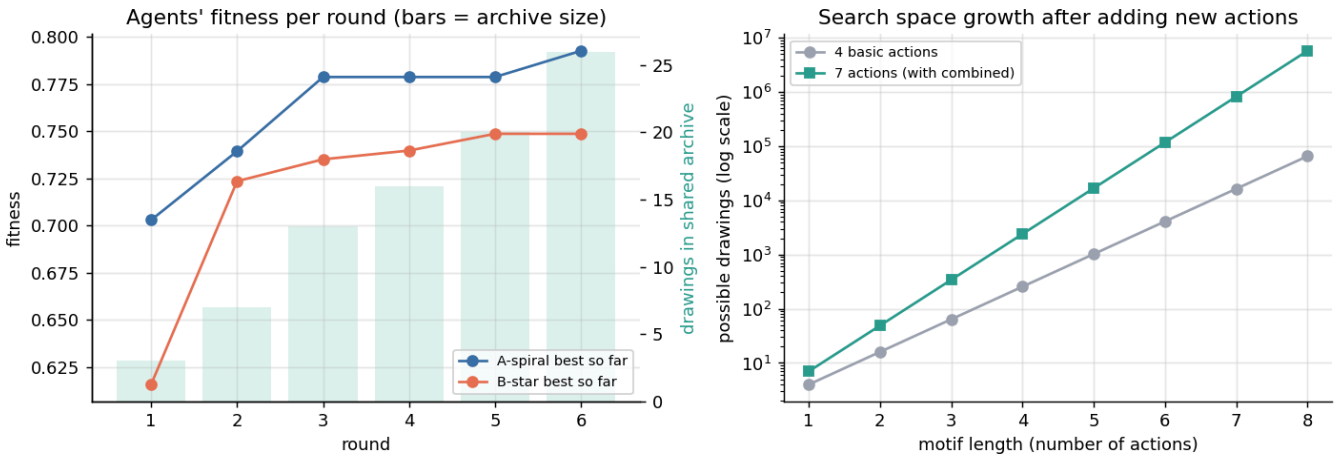


Figure 3. Left: each agent's best fitness per round, with archive size as bars. Right: search-space growth after adding new actions.

10. How do both agents create new states together?

- **Different styles.** A-spiral prefers blend; B-star prefers interleave. Their proposals come from different corners of the search space, which keeps diversity high. This is co-operative creativity, not two copies of one agent.
- **Shared archive.** Once one agent adds a drawing, the other gets no credit for repeating it. The pair is pushed toward fresh states.
- **Learning from each other.** Both agents ended up favouring "compose", because the archive and scores showed it worked.
- The final archive is a set of **choices, not one answer**. A human artist can pick the one they like.

11. Code and real run

The program `q08_code.py` draws the parents, runs both agents for 6 rounds and saves every figure in this answer.

File Edit View Run Terminal Help — q08_code.py - Q08_MultiAgent_Drawing_Patterns - Python IDE

q08_code.py

```python
AGENTS = {"A-spiral": ["compose", "blend", "nest"], "B-star": ["compose", "interleave", "nest"]}


def multi_agent_search(rounds: int = 6, proposals: int = 5) -> tuple:
    archive, history, all_cands = [], [], []  # archive = shared memory of new states
    best = {a: None for a in AGENTS}
    for r in range(1, rounds + 1):
        for agent, ops in AGENTS.items():
            for _ in range(proposals):
                if best[agent] and random.random() < 0.6:  # exploit: tweak own best idea
                    c = combine(best[agent]["op"], mutate(best[agent]["params"]))
                else:  # explore: brand-new combination of the two parents
                    c = combine(random.choice(ops), random_params())
                c = {**fitness(c), "agent": agent, "round": r}
                all_cands.append(c)
                if all(iou(c["grid"], g) < 0.6 for g in archive):  # a genuinely new state
                    archive.append(c["grid"])
                if best[agent] is None or c["fit"] > best[agent]["fit"]:
                    best[agent] = c
            history.append((r, agent, best[agent], len(archive)))
    return history, all_cands
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\Users\USER\questions\Answers\Q08_MultiAgent_Drawing_Patterns> python q08_code.py
Q8  Multi-agent creative drawing (Agent A = spiral, Agent B = star)
Action space: 4 basic actions -> 7 after 3 combined actions
Search space for a motif of 6 actions: 4^6 = 4,096 -> 7^6 = 117,649 (28.7x larger)
round | agent    | best operator | fitness | novelty | aesth | archive
    1 | A-spiral | compose       |   0.703 |   0.732 | 0.525 |       2
    1 | B-star   | nest          |   0.616 |   0.499 | 0.641 |       3
    2 | A-spiral | compose       |   0.739 |   0.646 | 0.702 |       5
    2 | B-star   | compose       |   0.724 |   0.681 | 0.628 |       7
    3 | A-spiral | compose       |   0.779 |   0.684 | 0.763 |      10
    3 | B-star   | compose       |   0.735 |   0.677 | 0.661 |      13
    4 | A-spiral | compose       |   0.779 |   0.684 | 0.763 |      16
    4 | B-star   | compose       |   0.740 |   0.667 | 0.683 |      16
    5 | A-spiral | compose       |   0.779 |   0.684 | 0.763 |      18
    5 | B-star   | compose       |   0.749 |   0.669 | 0.703 |      20
    6 | A-spiral | compose       |   0.793 |   0.672 | 0.810 |      23
    6 | B-star   | compose       |   0.749 |   0.669 | 0.703 |      26
Best drawing: compose by A-spiral (round 6), fitness 0.793
Parents scored aesthetics A = 0.540, B = 0.547
charts saved to images/
PS C:\Users\USER\questions\Answers\Q08_MultiAgent_Drawing_Patterns>
```

main — Ln 146, Col 1 — Spaces: 4 — UTF-8 — CRLF — Python 3.12.3

Figure 4. The multi-agent search function and the real terminal output.

12. Summary

> A search space is everything reachable through the available actions. Combining one action from each parent (compose, blend, nest, interleave) adds new actions. That grew the space 28.7 times and produced drawings neither parent could make. A fitness of novelty + aesthetics + balanced inheritance, plus a shared archive, guided two agents from fitness 0.62–0.70 to 0.75–0.79.

Files in this folder: `q08_code.py` (source), `q08_code.ipynb` (Colab/Jupyter version, already run), `q08_output.txt` (console output), `images/` (all figures).